Getting started with Elasticsearch in Python

Adnan Siddiqi · Follow
Published in Towards Data Science · 12 min read · Jun 2, 2018

2.6K 10



Image Credit: tryolabs.com

The updated version of this post for Elasticsearch 7.x is available here.

In this post, I am going to discuss Elasticsearch and how you can integrate it with different Python apps.

related features.

ElasticSearch Use Cases

You can use ES for multiple purposes, a couple of them given below:

- You are running a website that provides lots of dynamic content; be it an e-commerce website or a blog. By implementing ES you can not only provide a robust search engine for your web app but can also provide native auto-complete features in your app.
- You can ingest different kinds of log data and then can use to find trends and statistics.

Setting up and Running

The easiest way to install ElasticSearch is to just **download** it and run the executable. You must make sure that you are using Java 7 or greater.

Once download, unzip and run the binary of it.

```
elasticsearch-6.2.4 bin/elasticsearch
```

There will be a lots of text in the scrolling window. If you see something like below then it seems it's up.

```
[2018-05-27T17:36:11,744][INFO ][o.e.h.n.Netty4HttpServerTransport]
[c6hEGv4] publish_address {127.0.0.1:9200}, bound_addresses
```

```
{
  "name" : "c6hEGv4",
  "cluster_name" : "elasticsearch",
  "cluster_uuid" : "HkRyTYXvSkGvkvHX2Q1-oQ",
  "version" : {
    "number" : "6.2.4",
    "build_hash" : "ccec39f",
    "build_date" : "2018-04-12T20:37:28.497551Z",
    "build_snapshot" : false,
    "lucene_version" : "7.2.1",
    "minimum_wire_compatibility_version" : "5.6.0",
    "minimum_index_compatibility_version" : "5.0.0"
  },
  "tagline" : "You Know, for Search"
}
```

Now, before I move onto accessing Elastic Search in Python, let's do some basic stuff. As I mentioned that ES provides a **REST API**, we will be using it to carry on different tasks.

Basic Examples

The very first thing you have to do is creating an *Index*. Everything is stored in an Index. The RDBMS equivalent of *Index* is a *database* so don't confuse it with the typical indexing concept you learn in RDBMS. I am using **PostMan** to run REST APIs.

If it runs successfully you will see something like below in response.

```
{
    "acknowledged": true,
    "shards_acknowledged": true,
    "index": "company"
}
```

So we have created a database with the name *company*. In other words, we have created an *Index* called *company*. If you access `http://localhost:9200/company` from your browser then you will see something like below:

```
{
  "company": {
    "aliases": {

    },
    "mappings": {

    },
    "settings": {
      "index": {
        "creation_date": "1527638692850",
        "number_of_shards": "5",
        "number_of_replicas": "1",
        "uuid": "RnT-gXISSxKchyowgjZOkQ",
        "version": {
          "created": "6020499"
        },
        "provided_name": "company"
      }
    }
```

across them. In simple words, if there are 5 shards their entire data is available across 5 shards and ElasticSearch cluster can serve requests from any of its node.

Replicas talk about mirroring of your data. If you are familiar with the *master-slave* concept then this should not be new for you. You can learn more about basic ES concepts **here**.

The cURL version of creating an index is a one-liner.

```
➜  elasticsearch-6.2.4 curl -X PUT localhost:9200/company
{"acknowledged":true,"shards_acknowledged":true,"index":"company"}%
```

You can also perform both index creation and record insertion task in a single go. All you have to do is to pass your record in JSON format. You can something like below in *PostMan.*

Make sure you set `Content-Type` as `application/json`

It will create an index, named, `company` here if it does not exist and then create a new *type* called *employees* here. Type is actually the ES version of a *table* in RDBMS.

The above requests will output the following JSON structure.

```
{
    "_index": "company",
    "_type": "employees",
    "_id": "1",
    "_version": 1,
    "result": "created",
    "_shards": {
        "total": 2,
        "successful": 1,
        "failed": 0
    },
    "_seq_no": 0,
    "_primary_term": 1
}
```

You pass `/1` as an ID of your record. It is not necessary though. All it will do is to set `_id` field with the value `1`. You then pass your data in JSON format which will eventually be inserted as a new record or document. If you access

```
}}
```

You can see the actual record along with the meta. If you want you can change the request as `http://localhost:9200/company/employees/1/_source` and it will only output the JSON structure for the record only.

The cURL version would be:

```
➜  elasticsearch-6.2.4 curl -X POST \
>   http://localhost:9200/company/employees/1 \
>   -H 'content-type: application/json' \
>   -d '{
quote>     "name": "Adnan Siddiqi",
quote>     "occupation": "Consultant"
quote> }'
{"_index":"company","_type":"employees","_id":"1","_version":1,"resul
t":"created","_shards":
{"total":2,"successful":1,"failed":0},"_seq_no":0,"_primary_term":1}%
```

What if you want to update that record? Well, it's pretty simple. All you have to do is to change your JSON record. Something like below:

and it will generate the following output:

```
{
    "_index": "company",
    "_type": "employees",
    "_id": "1",
    "_version": 2,
    "result": "updated",
    "_shards": {
        "total": 2,
        "successful": 1,
        "failed": 0
    },
    "_seq_no": 1,
    "_primary_term": 1
}
```

Notice the `_result` field which is now set to `updated` instead of `created`

And of course, you can delete the certain record too.



And if you are going crazy or your girlfriend|boyfriend have dumped you, you can burn the entire world by running `curl -XDELETE localhost:9200/_all` from command-line.

Let's do some basic search. If you run `http://localhost:9200/company/employees/_search?q=adnan`, it will search all fields under the type `employees` and returns the relevant records.

```
{
  "took": 7,
  "timed_out": false,
  "_shards": {
    "total": 5,
    "successful": 5,
    "skipped": 0,
    "failed": 0
  },
  "hits": {
    "total": 1,
    "max_score": 0.2876821,
    "hits": [
      {
        "_index": "company",
        "_type": "employees",
        "_id": "1",
```

The `max_score` field tells how relevant the record is, that is, the highest score of the record. If there were multiple records then it'd be a different number.

You can also limit your search criteria to a certain field by passing the field name. Therefore, `http://localhost:9200/company/employees/_search?q=name:Adnan` will search only in `name` field of the document. It is actually SQL equivalent of `SELECT * from table where name='Adnan'`

I just covered the basic examples. ES can do lots of things but I will let you explore it further by reading the documentation and will switch over to accessing ES in Python.

Accessing ElasticSearch in Python

To be honest, the REST APIs of ES is good enough that you can use `requests` library to perform all your tasks. Still, you may use a **Python library** for ElasticSearch to focus on your main tasks instead of worrying about how to create requests.

Install it via pip and then you can access it in your Python programs.

```
pip install elasticsearch
```

To make sure it's correctly installed, run the following basic snippet from command-line:

Web Scraping and Elasticsearch

Let's discuss a little practical use case of using Elasticsearch. The objective is to access online recipes and store them in Elasticsearch for searching and analytics purpose. We will first scrape data from *Allrecipes* and store it in ES. We will also be creating a strict Schema or *mapping*, in case of ES, so that we can make sure that data is being indexed in correct format and type. I am just pulling the listing of salad recipes only. Let's begin!

Scraping data

```
import json
from time import sleep

import requests
from bs4 import BeautifulSoup

def parse(u):
    title = '-'
    submit_by = '-'
    description = '-'
    calories = 0
    ingredients = []
    rec = {}

try:
        r = requests.get(u, headers=headers)

if r.status_code == 200:
            html = r.text
            soup = BeautifulSoup(html, 'lxml')
            # title
            title_section = soup.select('.recipe-summary__h1')
            # submitter
            submitter_section = soup.select('.submitter__name')
            # description
            description_section =
soup.select('.submitter__description')
```

```
                    ingredient_text = ingredient.text.strip()
                    if 'Add all ingredients to list' not in
ingredient_text and ingredient_text != '':
                        ingredients.append({'step':
ingredient.text.strip()})

            if description_section:
                description =
description_section[0].text.strip().replace('"', '')

            if submitter_section:
                submit_by = submitter_section[0].text.strip()

            if title_section:
                title = title_section[0].text

            rec = {'title': title, 'submitter': submit_by, 'description':
description, 'calories': calories,
                   'ingredients': ingredients}
        except Exception as ex:
            print('Exception while parsing')
            print(str(ex))
        finally:
            return json.dumps(rec)

    if __name__ == '__main__':
        headers = {
            'User-Agent': 'Mozilla/5.0 (Macintosh; Intel Mac OS X
10_11_6) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/66.0.3359.181
Safari/537.36',
            'Pragma': 'no-cache'
        }
        url = 'https://www.allrecipes.com/recipes/96/salad/'
        r = requests.get(url, headers=headers)
        if r.status_code == 200:
            html = r.text
            soup = BeautifulSoup(html, 'lxml')
            links = soup.select('.fixed-recipe-card__h3 a')
            for link in links:
                sleep(2)
                result = parse(link['href'])
                print(result)
                print('================================')
```

So this is the basic program that pulls data. Since we need data in JSON format, therefore, I converted it accordingly.

Before we go to create an index, we have to connect ElasticSearch server.

```
import logging
def connect_elasticsearch():
    _es = None
    _es = Elasticsearch([{'host': 'localhost', 'port': 9200}])
    if _es.ping():
        print('Yay Connect')
    else:
        print('Awww it could not connect!')
    return _es

if __name__ == '__main__':
  logging.basicConfig(level=logging.ERROR)
```

`_es.ping()` actually pings the server and returns `True` if gets connected. It took me a while to figure out how to catch stack trace, **found out** that it was just being logged!

```
def create_index(es_object, index_name='recipes'):
    created = False
    # index settings
    settings = {
        "settings": {
            "number_of_shards": 1,
            "number_of_replicas": 0
        },
        "mappings": {
            "members": {
                "dynamic": "strict",
                "properties": {
                    "title": {
                        "type": "text"
                    },
                    "submitter": {
                        "type": "text"
                    },
                    "description": {
```

```
            if not es_object.indices.exists(index_name):
                # Ignore 400 means to ignore "Index Already Exist" error.
                es_object.indices.create(index=index_name, ignore=400,
body=settings)
                print('Created Index')
            created = True
        except Exception as ex:
            print(str(ex))
        finally:
            return created
```

Lot's of things happening here. First, we passed a config variable that contains the mapping of entire document structure. **Mapping** is the Elastic's terminology for a schema. Just like we set certain field data type in tables, we do something similar here. Check the docs, it covers more than that. All fields are of type `text` but `calories` which is of type `Integer`

Next, I am making sure that the index does not exist at all and then creating it. The parameter `ignore=400` is not required anymore after checking but in case you do not check the existence you can suppress the error and overwrite the existing index. It's risky though. It's like overwriting the DB.

If the index is successfully created, you can verify it by visiting **http://localhost:9200/recipes/_mappings** and it will print something like below:

```
{
  "recipes": {
    "mappings": {
      "salads": {
        "dynamic": "strict",
        "properties": {
          "calories": {
```

```
                }
            }
        }
    }
}
```

By passing `dynamic:strict` we are forcing Elasticsearch to do a strict checking of any incoming document. Here, `salads` is actually the document type. The `Type` is actually the Elasticsearch's answer of RDBMS *tables*.

Records Indexing

The next step is storing the actual data or document.

```
def store_record(elastic_object, index_name, record):
    try:
        outcome = elastic_object.index(index=index_name,
doc_type='salads', body=record)
    except Exception as ex:
        print('Error in indexing data')
        print(str(ex))
```

Run it and you will be welcomed by:

```
Error in indexing data
TransportError(400, 'strict_dynamic_mapping_exception', 'mapping set
to strict, dynamic introduction of [ingredients] within [salads] is
not allowed')
```

Can you guess why is it happening? Since we did not set `ingredients` in our mapping, ES did not allow us to store the document that contains

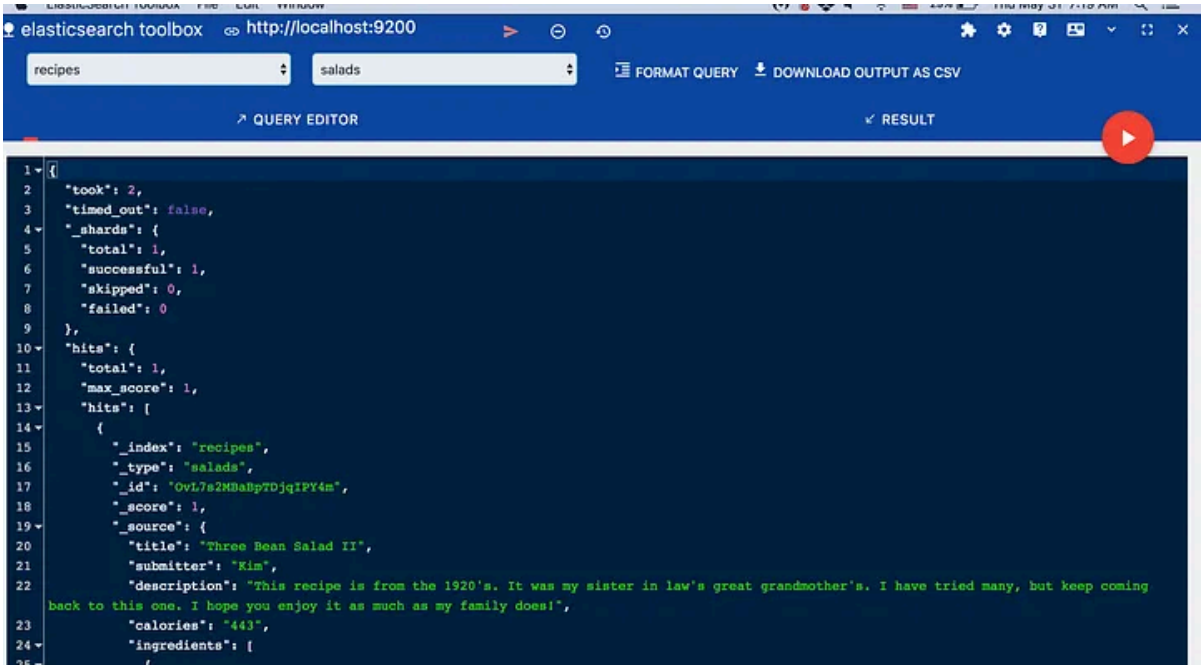
```
                "dynamic": "strict",
                "properties": {
                    "title": {
                        "type": "text"
                    },
                    "submitter": {
                        "type": "text"
                    },
                    "description": {
                        "type": "text"
                    },
                    "calories": {
                        "type": "integer"
                    },
                    "ingredients": {
                        "type": "nested",
                        "properties": {
                            "step": {"type": "text"}
                        }
                    },
                }
            }
        }
```

We added `ingrdients` of type `nested` and then assigned the data type of internal field. In our case it's `text`

The **nested** data type let you set the type of nested JSON objects. Run it again and you will be greeted by the following output:

```
{
  '_index': 'recipes',
  '_type': 'salads',
  '_id': 'OvL7s2MBaBpTDjqIPY4m',
  '_version': 1,
  'result': 'created',
  '_shards': {
    'total': 1,
    'successful': 1,
    'failed': 0
```

called **ElasticSearch Toolbox** to view the data.



Before we move on, let's send a string in `calories` field and see how it goes. Remember we had set it as `integer`. Upon indexing it gave the following error:

```
TransportError(400, 'mapper_parsing_exception', 'failed to parse
[calories]')
```

So now you know the benefits of assigning a mapping for your documents. If you don't, it will still work as Elasticsearch will assign its own mapping at runtime.

Querying Records

It is pretty basic. You pass index and search criteria in it. Let's try some queries.

```
if __name__ == '__main__':
  es = connect_elasticsearch()
    if es is not None:
        search_object = {'query': {'match': {'calories': '102'}}}
        search(es, 'recipes', json.dumps(search_object))
```

The above query will return all records in which `calories` is equal to 102. In our case the output would be:

```
{'_shards': {'failed': 0, 'skipped': 0, 'successful': 1, 'total': 1},
 'hits': {'hits': [{'_id': 'YkTAuGMBzBKRviZYEDdu',
                    '_index': 'recipes',
                    '_score': 1.0,
                    '_source': {'calories': '102',
                                'description': "I've been making
variations of "
                                               'this salad for years.
I '
                                               'recently learned how
to '
                                               'massage the kale and
it makes '
                                               'a huge difference. I
had a '
                                               'friend ask for my
recipe and I '
                                               "realized I don't have
one. "
                                               'This is my first
attempt at '
                                               'writing a recipe, so
please '
```

```
  instead of '                                                  'if eddy ever. Great as a
  side '
                                                              'dish or by itself the
  next day '
                                                              'for lunch!',
                                                 'ingredients': [{'step': '1 bunch
  kale, large '
                                                                          'stems
  discarded, '
                                                                          'leaves
  finely '
                                                                          'chopped'},
                                                                  {'step': '1/2
  teaspoon salt'},
                                                                  {'step': '1
  tablespoon apple '
                                                                          'cider
  vinegar'},
                                                                  {'step': '1 apple,
  diced'},
                                                                  {'step': '1/3 cup
  feta cheese'},
                                                                  {'step': '1/4 cup
  currants'},
                                                                  {'step': '1/4 cup
  toasted pine '
                                                                          'nuts'}],
                                                 'submitter': 'Leslie',
                                                 'title': 'Kale and Feta Salad'},
                                    '_type': 'salads'}],
                          'max_score': 1.0,
                          'total': 1},
   'timed_out': False,
   'took': 2}
```

What if you want to get records in which `calories` greater than 20?

```
search_object = {'_source': ['title'], 'query': {'range': {'calories':
{'gte': 20}}}}
```

apps searchable by providing robust features to return the most accurate result set. I have just covered the gist of it. Do read docs and get yourself acquainted with this powerful tool. Especially fuzzy search feature is quite awesome. If I get chance I will cover Query DSL in coming posts.

Like always, the code is available on **Github**.

This article was originally published here.

Click here to subscribe my newsletter for future posts.

Elasticsearch Python Data Analysis

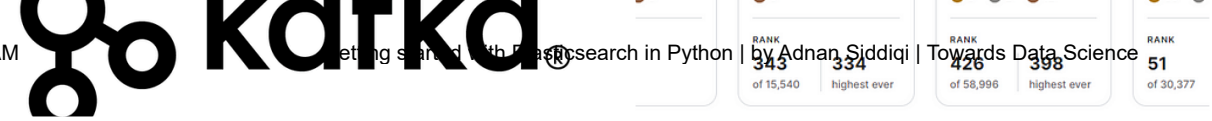


Written by Adnan Siddiqi

3K Followers · Writer for Towards Data Science

Pakistani | Husband | Father | Software Consultant | Developer | blogger. I occasionally try to make stuff with code. https://adnansiddiqi.me

Follow



Adnan Siddiqi in Towards Data Science

Getting started with Apache Kafka in Python

In this post, I am going to discuss Apache Kafka and how Python programmers can us...

9 min read · Jun 11, 2018

3.1K 13

Geremie Yeo in Towards Data Science

One Year of Consistent Kaggling: What Did It Teach Me?

Competitions are more valuable than other components

5 min read · May 11, 2024

1.6K 14



Almog Baku in Towards Data Science

Building LLM Apps: A Clear Step-By-Step Guide

Comprehensive Steps for Building LLM-Native Apps: From Initial Idea to...

12 min read · Jun 10, 2024

5.4K 14



Adnan Siddiqi in Towards Data Science

Getting started with Apache Cassandra and Python

In this post, I am going to talk about Apache Cassandra, its purpose, usage, configuration...

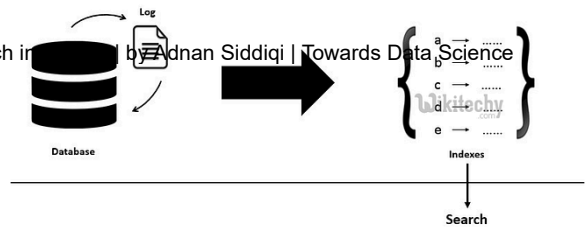
16 min read · May 19, 2019

316 5

See all from Adnan Siddiqi

See all from Towards Data Science





Yennhi95zz in Data And Beyond

How to Easily Automate Web Scraping with a Simple Data...

LLM for fully automated web scraping, creating an end-to-end data extraction...

· 9 min read · Jun 6, 2024

288

Navya Cloudops

Elastic Search—Day 2: Data Modeling and Indexing

Welcome to Day 2 of our 10-day DevOps Elasticsearch course! Yesterday, we delved...

· 9 min read · Feb 7, 2024

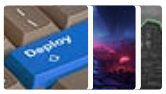
7 1

Lists



Coding & Development

11 stories · 652 saves



Predictive Modeling w/ Python

20 stories · 1288 saves



Practical Guides to Machine Learning

10 stories · 1548 saves



ChatGPT prompts

48 stories · 1671 saves

Wanna Code Like a FAANG Engineer? Together, let's dive into...

Learn about magical libraries, effective looping, syntax, and more. You absolutely...

12 min read · Jan 4, 2024

2.3K 15

Synonyms and Elasticsearch

Recently I was needing to search for a name and address using elasticsearch. I was trying...

2 min read · Mar 27, 2024



Abhishek Bairagi

Elasticsearch Tutorials Part 2: Installation, Setup, and Creating...

In the previous part of our Elasticsearch series, we explored the fundamentals and us...

6 min read · Dec 29, 2023

51



Max N in Level Up Coding

Build Python Search Engines with Elasticsearch

Unlock Powerful Text Analysis and ML Relevancy Ranking

2 min read · Feb 20, 2024

51

See more recommendations